

Relatório do LAB 5

1. Identificação

Nome: Lucas Hufnagel Gromann de Araujo Góes

Matrícula: 2410845

Nome: Caio Faria Brito Martins Chaves

Matrícula: 2410162

2. Objetivo

O objetivo deste laboratório é estudar e aplicar conceitos fundamentais de **sinais** e **processos** em sistemas operacionais, utilizando a linguagem C.

Foram explorados os seguintes tópicos:

- interceptação de sinais (SIGINT, SIGQUIT, SIGFPE e tentativa com SIGKILL);
- criação e controle de processos com fork;
- execução de programas com execvp e execlp;
- tratamento de exceções aritméticas;
- simulação de escalonamento de processos.

3. Estrutura do Programa

O trabalho foi dividido em 7 programas.

3.1 ctrl-c.c

Descrição:

Programa responsável por capturar sinais gerados pelo teclado:

- SIGINT (Ctrl+C);

- SIGQUIT (Ctrl+\).

Utiliza `signal()` para associar funções tratadoras aos sinais e executa um laço infinito imprimindo o PID do processo.

3.2 sigkill.c

Descrição:

Programa que tenta capturar o sinal SIGKILL por meio de um handler definido com `signal()`.

Em seguida, entra em loop infinito para permitir o teste.

3.3 loop.c

Descrição:

Programa auxiliar usado no escalonador.

Executa um loop infinito com `while(1)`, imprime o PID do processo e faz uma pausa de 1 segundo a cada repetição.

3.4 filhomicidio.c

Descrição:

Programa que cria um processo filho com `fork()` e executa diferentes comportamentos, dependendo da opção informada na linha de comando:

- (a) loop infinito;
- (b) `sleep(3)`;
- (c) `execvp("sleep", {"sleep", "5", NULL})`;
- (d) `execvp("sleep", {"sleep", "15", NULL})`.

O processo pai espera 2 segundos, envia SIGKILL ao filho e depois aguarda seu término com `waitpid()`. A versão final usa `execvp` nas opções com substituição de programa.

3.5 float.c

Descrição:

Programa que lê dois números reais e realiza as quatro operações básicas:

- soma;
- subtração;
- multiplicação;
- divisão.

Foi adicionada também a instalação de um tratador para SIGFPE, permitindo repetir o experimento e observar o comportamento da divisão por zero com números reais. Na prática, esse tipo de divisão normalmente não gera o mesmo comportamento observado com inteiros.

3.6 int.c

Descrição:

Programa que lê dois números inteiros e realiza as quatro operações básicas.

Também instala um tratador para SIGFPE, permitindo capturar o erro causado por divisão inteira por zero e exibir uma mensagem apropriada.

3.7 escalonador.c

Descrição:

Programa que cria dois processos filhos. Cada filho executa o programa `loop.c` com `exec`.

O processo pai realiza o controle da execução dos filhos usando:

- SIGSTOP para interromper;
- SIGCONT para retomar;
- SIGKILL para finalizar.

O pai alterna a execução dos dois processos, deixando cada um executar por 1 segundo, durante um total de 15 segundos. Ao final, os dois filhos são encerrados.

4. Solução

4.1 Questão 1 – ctrl-c.c

Com os sinais tratados

Ao executar o programa com os tratadores instalados:

- ao pressionar **Ctrl+C**, o processo recebe SIGINT, executa o handler e imprime a mensagem correspondente;
- ao pressionar ****Ctrl****, o processo recebe SIGQUIT, executa o handler e imprime a mensagem correspondente.

Nessa situação, o programa **não é encerrado**, pois o comportamento padrão dos sinais foi substituído pelos tratadores definidos com `signal()`.

Sem os comandos `signal()`

Ao remover as chamadas de `signal()` e repetir o experimento:

- **Ctrl+C** encerra o programa;
- ***Ctrl*** encerra o programa e pode gerar *core dump*, dependendo do sistema.

Explicação

O uso de `signal()` permite interceptar sinais e alterar seu comportamento padrão. Sem os tratadores, o sistema operacional aplica a ação padrão associada a cada sinal.

4.2 Questão 2 – tentativa de capturar SIGKILL

Resultado

Não foi possível capturar o sinal SIGKILL.

Explicação

O sinal SIGKILL é um sinal especial do sistema operacional. Ele **não pode ser interceptado, ignorado nem tratado** por programas de usuário. Sua função é garantir que o sistema consiga encerrar um processo de forma imediata e obrigatória.

Assim, embora o programa tente registrar um handler para SIGKILL, essa tentativa não produz o efeito desejado.

4.3 Questão 3 – filhomicidio.c

O programa cria um processo filho e, após 2 segundos, o pai tenta encerrá-lo com SIGKILL. O que acontece depende do comportamento executado pelo filho.

a) Filho em loop infinito

O filho entra em laço infinito e nunca termina sozinho.

Após 2 segundos, o pai envia SIGKILL, e o filho é encerrado com sucesso.

b) Filho executando sleep(3)

O filho fica dormindo por 3 segundos.

Como o pai envia SIGKILL após apenas 2 segundos, o filho ainda está em execução e é morto antes de terminar o sleep.

c) Filho executando sleep 5 com execvp

O filho substitui sua imagem por um novo programa (sleep 5).

Mesmo sendo outro programa, o PID continua sendo o mesmo. Após 2 segundos, o pai envia SIGKILL, e o processo é encerrado antes do término do sleep.

d) Filho executando sleep 15 com execvp

O comportamento é análogo ao caso anterior, mas com duração maior.

O pai também consegue matá-lo antes do término.

Conclusão

O experimento mostra que:

- fork() cria um novo processo filho;
- execvp() substitui o código executado pelo filho, mas mantém o mesmo processo;
- SIGKILL encerra o filho independentemente de ele estar em loop, dormindo ou executando outro programa.

4.4 Questão 4 – operações matemáticas

Operações com números reais

O programa com números reais realiza normalmente as quatro operações.

Quando o segundo valor digitado é zero, a divisão em ponto flutuante geralmente **não provoca encerramento imediato do programa**. Dependendo do sistema e da implementação, o resultado pode ser:

- inf;
- -inf;
- nan.

Mesmo com a instalação de um handler para SIGFPE, o comportamento com reais pode diferir do comportamento com inteiros, pois operações em ponto flutuante seguem regras específicas do padrão IEEE 754.

Operações com inteiros

No caso dos inteiros, quando o segundo valor é zero, a divisão provoca erro aritmético e o processo recebe SIGFPE.

Como o programa instala um tratador para esse sinal, ele captura o erro e exibe uma mensagem em vez de simplesmente abortar.

Conclusão

A principal diferença observada foi:

- com **inteiros**, divisão por zero gera SIGFPE;
- com **reais**, divisão por zero tende a produzir um valor especial (inf ou nan) em vez de encerrar imediatamente o processo.

4.5 Questão 5 – escalonador

Funcionamento

O programa cria dois processos filhos, e ambos executam um programa em loop infinito.

Logo após a criação, o pai interrompe os dois filhos com SIGSTOP. Em seguida, passa a coordenar a execução deles, alternando entre os dois processos:

- o filho 1 executa por 1 segundo;
- depois é interrompido com SIGSTOP;
- o filho 2 executa por 1 segundo;
- depois também é interrompido.

Esse procedimento se repete até completar 15 segundos de execução total do escalonador. Ao final, o pai envia SIGKILL aos dois filhos e encerra a execução.

Preempção

A preempção foi simulada com o uso dos sinais:

- SIGSTOP → interrompe a execução do processo;
- SIGCONT → permite que o processo volte a executar.

Dessa forma, o pai atua como um pequeno escalonador, controlando qual processo pode usar a CPU em cada instante.

Dificuldades encontradas

As principais dificuldades foram:

- entender a diferença entre criar processos (fork) e substituir o programa executado (exec);
- coordenar corretamente os sinais enviados aos filhos;
- garantir que o tempo total de execução do escalonador fosse de 15 segundos;
- controlar o encerramento correto dos processos ao final da execução.

Resultado

O programa funcionou de acordo com o esperado, alternando a execução dos dois filhos e demonstrando, na prática, uma forma simples de escalonamento cooperativo controlado pelo processo pai.

5. Observações e Conclusões

Dificuldades

Durante o laboratório, as principais dificuldades encontradas foram:

- compreender o comportamento dos sinais no sistema operacional;
- diferenciar sinais tratáveis e não tratáveis;
- trabalhar com fork, exec e sincronização entre pai e filho;
- controlar corretamente processos simultâneos.

Facilidade

Entre os pontos mais simples do laboratório, destacam-se:

- uso de funções padrão, como signal(), sleep(), kill() e waitpid();
- organização modular dos programas;
- possibilidade de testar cada caso separadamente.

Resultados dos testes

Os testes mostraram que:

- a captura de SIGINT e SIGQUIT funcionou corretamente;
- SIGKILL não pôde ser capturado;
- o tratamento de SIGFPE funcionou no caso de divisão inteira por zero;
- a execução com `execvp` substituiu corretamente o programa executado pelo filho;
- o escalonador alternou corretamente a execução entre os dois processos filhos.

Conclusão final

O laboratório permitiu compreender, de forma prática, o funcionamento de conceitos importantes de sistemas operacionais, especialmente:

- sinais;
- criação e controle de processos;
- tratamento de erros aritméticos;
- escalonamento e preempção.

A implementação do escalonador foi o ponto mais relevante do trabalho, pois demonstrou na prática como um processo pode controlar a execução de outros por meio de sinais.

Como possíveis melhorias futuras, podem ser citadas:

- implementação de prioridades entre processos;
- inclusão de mais políticas de escalonamento;
- tratamento mais robusto de erros;
- ampliação dos testes com outros sinais e comportamentos.